

STRINGS & LISTS

Programming 2

Presentation Outline

- String Class
- String Properties & Methods
- Verbatim String Literal
- String Builder
- String versus String Builder
- List Properties & Methods

03

What is String Class

In C#, string (lowercase) is a keyword that refers to the System.String class in the .NET Framework.

When you code:

```
string name = "Juan";
```

Actually the same as:

```
System.String name = "Juan";
```

04

Key Characteristics of the String Class

01

Immutable

- Once a string is created, it cannot be changed.
- Any operation that modifies the string actually returns a new string.

02

Supports Indexing

- You can access individual characters using index numbers:

03

Belongs to the System Namespace

You need **using System;** at the top of your program to use the class without writing System.String.

05

Key Characteristics of the String Class

01

Immutable

```
string original = "Hello";
string modified = original.Replace("H", "J");
Console.WriteLine(original);  // Hello
Console.WriteLine(modified);  // Jello
```

02

Supports Indexing

```
string word = "Hello";
char firstLetter = word[0];  // H
```

06

Common Properties of String

Properties	Description
Length	Returns the number of characters
Char[i]	Gets the character at index i

07

Common Properties of String

Sample Code

```
string greeting = "Hi!";
Console.WriteLine(greeting.Length);
// 3
Console.WriteLine(greeting[1]);       // i
```

08

Common Methods in String Class

Method	Description
ToUpper()	Converts to uppercase
ToLower()	Converts to lowercase
Substring(start, len)	Extracts a portion of the string
IndexOf("x")	Finds the position of a character
Replace("a", "b")	Replaces characters
Trim()	Removes spaces at start and end
Split(' ')	Splits into an array of strings
Contains("text")	Checks if text exists inside the string

09

Common Properties of String

Sample Code

```
string sentence = "  C# is Powerful!  ";
Console.WriteLine(sentence.Trim());             // "C# is Powerful!"
Console.WriteLine(sentence.ToUpper());          // "  C# IS POWERFUL!  "
Console.WriteLine(sentence.Contains("Power")); // True
Console.WriteLine(sentence.Substring(5, 2));   // "is"
```

What is a Verbatim String Literal?

A verbatim string literal is a string that ignores escape sequences like \\ or \n and preserves the text exactly as written, including newlines and backslashes.

Verbatim String

Syntax:

You write a verbatim string literal by prefixing the string with @:

```
string path = @"C:\Users\Juan\Documents\Project";
```

Without @, you'd have to write:

```
string path = "C:\\Users\\Juan\\Documents\\Project";
```

When to use verbatim string

1. File paths
2. Multiline strings
3. Strings with lots of escape characters

```
string filePath =
@"D:\Projects\MyApp\code.cs";
Console.WriteLine(filePath);
```

Equivalent to:

```
string filePath =
"D:\\Projects\\MyApp\\code.cs";
```

Multiline String

Code:

```
string letter = @"Dear Student,

Thank you for submitting your
project.
Keep up the good work!

Regards,
Your Instructor";
Console.WriteLine(letter);
```

Output

```
Dear Student,

Thank you for submitting your
project.
Keep up the good work!

Regards,
Your Instructor
```

String Builder

What is String Builder

The StringBuilder class (from the System.Text namespace) is used to create and modify strings without creating new objects each time.

Unlike string, which is immutable, StringBuilder is mutable — you can change its content without creating a new object.

When Should You Use StringBuilder?

When your’re:

- Concatenating strings in a loop
- Doing lots of insertions, deletions, or replacements
- Improving performance in large-scale string operations

Sample:

```
using System;
using System.Text;

class Program
{
    static void Main()
    {
        StringBuilder sb = new StringBuilder();

        sb.Append("Hello");
        sb.Append(" ");
        sb.Append("World");

        Console.WriteLine(sb.ToString());
         // Output: Hello World
    }
}
```

16

Common StringBuilder Methods

01

Append(String)

- Adds text to the end

02

Index(index, string)

- Inserts text at a specific index

03

Remove(start, length)

- Removes characters

04

Replace(old, new)

- Replaces a substring

05

Clear()

- Removes all content

06

ToString()

- Converts the builder into a regular string

Sample

```
StringBuilder sb = new StringBuilder("Welcome");

sb.Append(" to C#");
sb.Insert(0, ">> ");
sb.Replace("C#", "CSharp");
sb.Remove(0, 3);  // Remove ">> "

Console.WriteLine(sb.ToString());  // Output: Welcome to CSharp
```

Note:

Use StringBuilder for heavy string operations.
Saves memory and processing time when modifying strings multiple times.
Don't forget to ToString() when you want the final result.

String vs StringBuilder

Performance Comparison

🔍 Performance Comparison

Operation	`string`	`StringBuilder`
Modify in loop	❌ Slower	✅ Faster
Immutable	✅ Yes	❌ No (it's mutable)
Use in light ops	✅ Fine	❌ Overkill



List Properties & Methods

What is a List in C#?

A List<T> is a generic collection in C# that allows you to store a dynamic array of objects.

It's part of the System.Collections.Generic namespace.

You can add, remove, access, and manipulate elements without worrying about fixed size (unlike arrays).

Syntax

```
List<int> numbers = new List<int>();
List<string> names = new List<string>();
```

Common Properties of a List

01

Count

- Number of elements in the list

02

Capacity

- Number of elements the list can hold before resizing

03

Item[index]

- Gets/sets element at specific index

Common Methods of a List

01

Add(Item)

- Adds an item to the list

02

AddRange(collection)

- Adds multiple items from another list

03

Insert(index, item)

- Inserts an item at a specific index

04

Remove(item)

- Removes the first occurrence

05

RemoveAt(

- Removes the item at a specific index

06

Clear()

- Removes all items

Sample

```
List<string> colors = new List<string>();

colors.Add("Red");
colors.Add("Blue");
colors.Add("Green");
colors.Insert(1, "Yellow");  // Insert at index 1

colors.Remove("Red");        // Remove "Red"
colors.Sort();               // Sort the list alphabetically

foreach (string color in colors)
{
    Console.WriteLine(color);
}
```

Next Topic: Struct in User-Defined Functions & List in User-Defined Functions